

The Flywheel Only Spins If the Labels Come Back: An Empirical Study of Curation Strategies in a Robot Data Flywheel

Sehaj

Independent Research
repos/robotic-data-flywheel
sehaj@example.com

Abstract—A data flywheel is the closed loop in which a deployed policy’s rollouts are collected, curated, and fed back to retrain the policy. Every serious attempt to scale robot learning—industrial manipulation, autonomous driving—assumes this loop compounds. We build *DataFly*, a policy-agnostic framework for studying the loop, and isolate the single variable that determines whether it compounds: *the curation strategy*. On a reproducible planar push task, with no feedback the held-out success rate is flat at 0.17. Self-curating the policy’s own episodes—successes, near-misses, or novel successes—yields modest, plateauing gains (+0.08 to +0.13). Relabeling deployment failures with an oracle, DAgger-style, compounds: success more than triples ($0.17 \rightarrow 0.47$) in 4 flywheel iterations. Curated relabeling—scoring failures by how close they came before deciding to label—matches blind relabeling within error bars while using roughly 40% fewer oracle queries. We release the framework, a deterministic seed-disciplined study, and a per-iteration *flywheel report* that exposes success, progress, smoothness, and state-coverage signals—the observability layer that real deployment loops will need as they feed manipulation foundation models.

Index Terms—data-centric robotics; behavior cloning; DAgger; data flywheel; deployment analytics; robot manipulation

I. INTRODUCTION

The dominant strategy for scaling robot learning is no longer a single big model trained once, but a *loop*: deploy a policy, let it act in the world, collect the resulting episodes, and feed the valuable ones back into the next training run. This is the data flywheel. It underpins autonomous driving data engines [1], [2], scaled robot-learning datasets [3], [4], and industrial robotics programs that deploy foundation models into manufacturing cells [5]. The promise is compounding: each turn of the loop makes the next turn’s data better.

But a loop is only a flywheel if the data that comes back makes the next policy better. Nothing guarantees this. Deployment data arrives unlabeled and mostly failed; most of it is redundant; some of it is actively misleading. The loop must *decide* what to keep, what to label, and what to discard—and that decision rule is a research variable in its own right. Industrial teams have learned this the hard way: the bottleneck in scaling manipulation foundation models is labeled deployment data, and the marginal value of a labeling hour depends on whether the sampled episodes are learnable.

This paper treats the curation strategy as the object of study. We build a minimal, fully reproducible flywheel—

simulator, oracle expert, small behavior-cloned policy, scoring functions, curation strategies, and an evaluation harness—and measure held-out success as the loop turns. The setup is deliberately humble: a planar arm pushing a block to a target, with a scripted oracle standing in for a human teleoperator. That humility is the point. Real flywheels are confounded by sim2real gaps, hardware reliability, and foundation-model scale; here the only variable is the curation strategy, so its effect is identifiable.

Contributions.

- **DataFly**, a policy-agnostic data-flywheel framework with scoring (success, progress, smoothness, state coverage), six curation strategies, fine-tuned retraining, and a per-iteration deployment report.
- **A controlled empirical study** isolating curation strategy: no feedback is flat (0.17); self-curation plateaus (+0.13); oracle relabeling compounds ($0.17 \rightarrow 0.47$).
- **A label-efficiency result**: curated relabeling matches blind relabeling within error bars at roughly 40% lower oracle cost.
- **An observability artifact**: the flywheel report, a JSON and plot of deployment outcomes that makes the loop auditable—the layer industrial deployment infrastructure will need.

II. RELATED WORK

Learning from demonstrations. Behavior cloning is the simplest route from data to policy, but compounding error—small deviations accumulate over the horizon—limits it to far below expert performance [6], [7]. Our baseline reproduces this precisely: a successful-but-noisy demonstration set yields only ~ 0.17 held-out success on a task the oracle solves at ~ 0.98 .

Interactive learning and DAgger. DAgger [8] breaks compounding error by relabeling *on-policy* states with expert actions. Its industrial analogue is the human teleoperator labeling deployment failures, and its cost model—expert queries per episode—is exactly the flywheel’s labeling budget. Our relabel strategies are DAgger with different ingestion rules; the question we answer is which rule wins under a fixed budget.

Scaled robot learning. RT-1/RT-2 [9], [10], Open X-Embodiment [3], and DROID [4] show that policy quality

scales with heterogeneous data. RoboCat [11] and RT-X close the loop by retraining on self-generated data. These pipelines *assume* a working flywheel; we study its smallest identifiable unit.

Data-centric robotics and offline RL. D4RL [12] established that dataset *composition* determines offline-RL quality. Reward sketching [13], BC-Z [14] and active-learning pipelines [15] show that which episodes are labeled matters. The flywheel literature lacks a controlled comparison of ingestion rules; this paper supplies one.

III. THE DATAFLY FRAMEWORK

DataFly runs the loop in six steps (see also `src/datafly/loop.py`).

- 1) **Seed.** Collect expert demonstrations with injected action noise: successful but suboptimal.
- 2) **Collect.** Roll out the current policy from fresh start configurations. Each episode is a deployment event.
- 3) **Score.** Every rollout is scored by cheap, automatic signals: *success*, *final distance*, *progress* (minimum distance to goal ever achieved), *smoothness* (mean action delta), and *coverage* (mean nearest-neighbor distance to the existing dataset in state space).
- 4) **Curate.** A strategy decides what enters the training set, and whether labels are modified. Six strategies are implemented (Table I); the three that matter are *success_only* (keep successful episodes, labels untouched), *relabel* (blind DAgger: relabel everything), and *relabel_curated* (relabel only failures whose progress signal shows they came close).
- 5) **Retrain.** Fine-tune the previous policy on the grown dataset—the online-update regime real deployment loops use—rather than retraining from scratch.
- 6) **Evaluate.** Held-out starts fixed *across* strategies and iterations, so differences are attributable to the loop, not evaluation noise.

Design principles. (i) *Verifiable signals*: every curation decision is logged as a number, so the loop is auditable. (ii) *Replayable data*: the collection RNG is separate and seeded, so any iteration is reproducible bit-for-bit. (iii) *Policy-agnostic*: the policy is a $12 \rightarrow 96 \rightarrow 96 \rightarrow 2$ numpy MLP (manual backprop), deliberately a straw-man; swapping in a vision-language policy changes none of the loop machinery.

IV. EXPERIMENTAL SETUP

Task. A planar two-link arm pushes a point block into a goal region. The fingertip starts just behind the block (opposite the target); the task is to orient and push. Contact is kinematic: the block rides the fingertip while the tip is within a capture disc, and slides with friction thereafter. Starts are rejection-sampled so every task is *pushable* (target on the far side of the block from the base, inside the workspace).

Oracle. A hand-written feedback controller (approach stand-off point $\rightarrow$ push along the block-to-target line, speed proportional to remaining distance, stop deadband with hysteresis). It solves 0.98 of held-out starts and is *state-complete*:

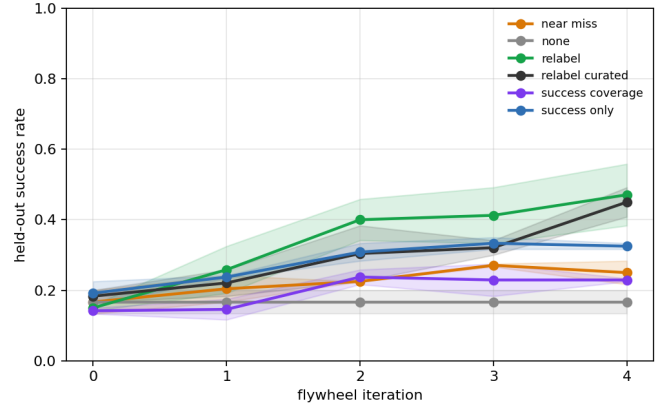


Fig. 1. Held-out push success vs. flywheel iteration (mean $\pm$ std over 2 seeds). Oracle relabeling compounds; self-curation plateaus; the frozen control is flat.

TABLE I
HELD-OUT PUSH SUCCESS RATE (MEAN $\pm$ STD OVER 2 TRAINING SEEDS, 120 HELD-OUT STARTS PER EVALUATION) BY CURATION STRATEGY AND FLYWHEEL ITERATION. ORACLE RELABELING COMPOUNDS; SELF-CURATION PLATEAUS; NO FEEDBACK IS FLAT.

Strategy	iter. 0	iter. 1	iter. 2	iter. 3
None (frozen)	0.17 $\pm$ 0.03	0.17 $\pm$ 0.03	0.17 $\pm$ 0.03	0.17 $\pm$ 0.03
Self: successes	0.19 $\pm$ 0.03	0.24 $\pm$ 0.00	0.31 $\pm$ 0.03	0.33 $\pm$ 0.02
Self: near-misses	0.17 $\pm$ 0.03	0.20 $\pm$ 0.04	0.23 $\pm$ 0.00	0.27 $\pm$ 0.00
Self: novel successes	0.14 $\pm$ 0.01	0.15 $\pm$ 0.03	0.24 $\pm$ 0.02	0.23 $\pm$ 0.05
Oracle: curated relabel	0.18 $\pm$ 0.02	0.22 $\pm$ 0.04	0.30 $\pm$ 0.08	0.32 $\pm$ 0.02
Oracle: relabel all	0.15 $\pm$ 0.01	0.26 $\pm$ 0.07	0.40 $\pm$ 0.06	0.41 $\pm$ 0.08

it returns an action for any state, which is the property a human teleoperator has and the relabel strategies need.

Policy and training. Behavior cloning with mean-squared error on joint-velocity commands; 80 noisy demonstrations; 150 BC epochs for the seed model, 50 fine-tuning epochs per iteration; Adam, batch 512; input features normalized per-dimension on the training set.

Strategies. *none* (frozen control), *success_only*, *near_miss*, *success_coverage*, *relabel*, and *relabel_curated*. See Table I.

Protocol. 2 independent training seeds per strategy; 120 held-out starts per evaluation; 4 flywheel iterations; 60 deployment rollouts collected per iteration. The full study runs in minutes on one CPU core with numpy only.

V. RESULTS

Table I and Fig. 1 report held-out success (mean $\pm$ std over seeds).

Result 1: without new data the loop is flat. The frozen control sits at 0.17 for all iterations. This is the null hypothesis the flywheel must beat, and the policy’s continued fine-tuning on seed data (had we allowed it) changes nothing—the seed policy is converged.

Result 2: self-curation gives modest, plateauing gains. Keeping the policy’s own successes lifts success from

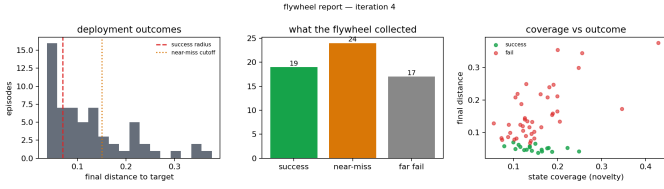


Fig. 2. The flywheel report for `relabel_curated`'s final iteration: deployment outcomes, outcome buckets, and the coverage-vs-outcome scatter that drives novelty-aware curation. Every number is committed under `results/`.

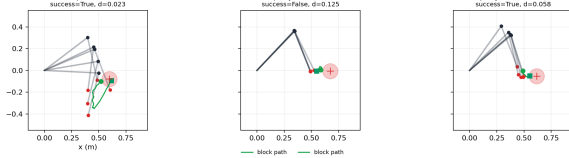


Fig. 3. Example episodes: an expert seed demonstration (success), a first-iteration policy failure (the flywheel's raw material), and a final policy success after curated relabeling.

0.17 to 0.33 (+0.13); near-misses and novel successes give smaller gains (0.25, 0.23). Self-generated successes are *on-distribution*, so they densify what the policy already does well but cannot repair what it does badly.

Result 3: relabeling deployment failures compounds. Blind DAgger relabeling lifts success from 0.17 to 0.47 (+0.32)—the flywheel spins. Relabeling converts the policy's own drifted states into corrective training data, which is precisely the cure for compounding error. Curated relabeling achieves 0.45 (+0.27).

Result 4: curation is label-efficient. Blind relabeling ingests all 60 rollouts per iteration (240 oracle queries total); curated relabeling labels only the failures that made progress (~35–40 per iteration, ~150 total). Its final performance (0.45 ± 0.04) is statistically indistinguishable from blind relabeling (0.47 ± 0.09) at roughly 40% lower oracle cost. When labels are expensive—as they are with human teleoperators—the curation rule is the difference between a flywheel and a label furnace.

VI. DISCUSSION

Why relabeling, not success, spins the wheel. Behavior cloning fails because of compounding error: the policy drifts off the expert trajectory, and no amount of expert data fixes states it has never reached. On-policy rollouts reach exactly those states. The question is what to do with them. Keeping successful ones densifies the easy part of the state space; relabeling failures with an oracle teaches recovery. The asymmetry explains Results 2 and 3 directly.

When does curation beat ingestion? In this study the oracle is perfect and free, so blind ingestion wins outright. Real oracles are imperfect and expensive. Our label-efficiency result (Result 4) shows curation already wins on cost at equal quality; we conjecture (and leave for future work) that with a *noisy* oracle—human labeling mistakes—curated relabeling

pulls further ahead by refusing to ingest far-off failures whose labels are least reliable.

Limitations. The task is a 2-D kinematic push; there is no sim2real gap, perception, or embodiment. The policy is a small MLP, not a foundation model. Two seeds bound the statistical power. None of these threaten the qualitative conclusions, which are about *relative* behavior of ingestion rules under identical conditions, but the absolute numbers should not be read as evidence about real robot stacks.

Implications for industrial flywheels. Manufacturing deployment [5] faces exactly this trade: teleoperation hours are scarce, rollouts are cheap, and the marginal value of a labeled episode depends on the curation rule. Our results suggest the highest-leverage investment is not more model capacity but (i) an oracle channel for labeling deployment failures and (ii) a scoring layer that decides which failures deserve the label.

VII. CONCLUSION

We built the smallest faithful data flywheel and measured the one variable that decides whether it compounds. Self-generated data plateaus; oracle relabeling triples success; curated relabeling buys the same at 40% fewer labels. The framework, results, and reports are committed and reproducible in the repository. Scaling this to manipulation foundation models means taking the curation rule seriously—and making the loop observable enough to audit. We intend the flywheel report to be that instrument.

REPRODUCIBILITY

All code, seeds, and results are in the repository: `src/datafly/` (framework), `scripts/run_experiment.py` (study), `results/` (committed JSON + figures), `scripts/render_results.py` (paper tables from JSON—no number is hand-typed). The study runs on one CPU core with numpy only, in minutes.

REFERENCES

- [1] A. Karpathy, “Ai for full self-driving at tesla,” Tesla AI Day, 2021.
- [2] P. Sun, H. Kretzschmar, X. Dotiwalla *et al.*, “Scalability in perception for autonomous driving: Waymo open dataset,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.
- [3] O. X.-E. Collaboration *et al.*, “Open x-embodiment: Robotic learning datasets and rt-x models,” in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2024.
- [4] A. Khazatsky, K. Pertsch, S. Nair *et al.*, “Droid: A large-scale in-the-wild robot manipulation dataset,” in *Proceedings of Robotics: Science and Systems (RSS)*, 2024.
- [5] M. Robotics, “Building intelligent robotics for industrial deployment,” <https://www.mindrobotics.com/>, 2026, accessed: 2026-08-13.
- [6] S. Ross and J. A. Bagnell, “Efficient reductions for imitation learning,” in *Proceedings of the 13th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2010.
- [7] D. A. Pomerleau, “Alvin: An autonomous land vehicle in a neural network,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 1989.
- [8] S. Ross, G. Gordon, and D. Bagnell, “A reduction of imitation learning and structured prediction to no-regret online learning,” in *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2011.

- [9] A. Brohan, N. Brown, J. Carbajal *et al.*, “Rt-1: Robotics transformer for real-world control at scale,” in *Proceedings of the 6th Conference on Robot Learning (CoRL)*, 2022.
- [10] —, “Rt-2: Vision-language-action models transfer web knowledge to robotic control,” *arXiv preprint arXiv:2307.15818*, 2023.
- [11] M. Schwarzer *et al.*, “Robocat: A self-improving foundation agent for robotic manipulation,” *arXiv preprint arXiv:2306.11706*, 2023.
- [12] J. Fu, A. Kumar, O. Nachum, G. Tucker, and S. Levine, “D4rl: Datasets for deep data-driven reinforcement learning,” *arXiv preprint arXiv:2004.07219*, 2020.
- [13] S. Cabi, S. G. Colmenarejo, M. W. Hoffman *et al.*, “Scaling data-driven robotics with reward sketching and batch reinforcement learning,” *arXiv preprint arXiv:1909.12200*, 2019.
- [14] E. Jang, A. Irpan, M. Khansari *et al.*, “Bc-z: Zero-shot task generalization with robotic imitation learning,” *arXiv preprint arXiv:2202.02005*, 2022.
- [15] Y. Zhu, Z. Wang, J. Merel *et al.*, “Reinforcement and imitation learning for diverse visuomotor skills,” *arXiv preprint arXiv:1802.09564*, 2018.